

ADAPTIVE COGNITIVE-LOAD STUDY TIMER FOR PERSONALIZED BREAK SCHEDULING

Project ID: 25-26J-458

Boghawatta B P S - IT22148254

Project Proposal Report

B.Sc. (Hons) Degree in Information Technology

**Department of Information Technology
Sri Lanka Institute of Information Technology
Sri Lanka**

August 2025

TABLE OF CONTENTS

Declaration	3
Abstract	4
List of Figures	4
List of Tables.....	5
List of Abbreviations.....	Error! Bookmark not defined.
1 Introduction	5
2 Background and Literature Survey	7
2.1 Cognitive Load and Productivity	7
2.2 Ecological momentary assessment and micro-EMA	7
2.3 Passive sensing via keystroke and touch dynamics	9
2.4 Privacy considerations	9
2.5 Current study tools and gaps	9
3 Research Gap and Problem Statement	12
3.1 Research Gap and Problem Statement	12
3.2 Problem Statement	13
4 Objectives.....	14
4.1 Main Objective.....	14
4.2 Specific Objectives.....	14
5 Methodology	15
5.1 Research Design.....	15
5.2 Data Collection.....	15
5.3 Feature Extraction and Modelling.....	19
5.4 Implementation	19
5.5 Evaluation Plan	22
6 Description of personal and Facilities.....	22
Personal Roles and Expertise	Error! Bookmark not defined.
Facilities and Equipment.....	Error! Bookmark not defined.
7 Work Breakdown Structure (WBS)	23
8 Self-Evaluation Plan.....	25
9 Budget	25
10 Commercialization Potential.....	26
11 Conclusion	26

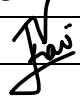
12 References	27
---------------------	----

Author: - Name Kaluarachchi V.I. ID: IT22148254

Supervisor: Dr. Kalpani Manathunga

Declaration

We declare that this is our own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	Student ID	Signature
Bogahawatta B.P.S.	IT22054418	

Supervisor Statement

The above candidate is carrying out research for the undergraduate dissertation under my supervision.

Signature of the supervisor:

Date:

Abstract

Fixed-schedule study timers such as the Pomodoro method assume that all learners have the same cognitive endurance, yet research demonstrates that cognitive load fluctuates significantly within and across study sessions. Our subsystem introduces an Adaptive Scheduler driven by a Contextual Bandit algorithm. The bandit dynamically chooses work interval lengths (e.g., 20/30/45/60 minutes) and break durations (e.g., 3/5/8/12 minutes) based on the learner's real-time cognitive state. Unlike fixed schedules, the scheduler personalises cycles by balancing two rewards: task progress and post-break relief. Novel metrics are introduced, including Regret-per-Hour (RPH), Personalization Gain (PG), Adaptation Half-Life (AHL), Exploration Overhead Index (EOI), Break Utility Curve (BUC) with AUC-BUC, Counterfactual Targeting Uplift (CTU), and Stability–Productivity Frontier (SPF). These metrics capture efficiency, adaptation speed, and stability beyond traditional cumulative regret. By combining contextual bandit methods with productivity-focused metrics, this subsystem advances personalised learning technologies.

List of Figures

1. **Figure 2.1:** Conceptual architecture of the dynamic scheduler
2. **Figure 2.2:** Comparison of EMA and passive sensing on burden and sampling frequency

List of Tables

1. **Table 5.1:** Summary of features used for cognitive load estimation
2. **Table 5.2:** Evaluation of metrics and targets
3. **Table 7.1:** Work breakdown structure (WBS)

1 Introduction

Studying and knowledge work are cognitively demanding tasks that require careful management of attention, energy, and recovery. Traditional productivity aids such as the Pomodoro timer enforce rigid cycles of work and break (e.g., 25 minutes of work followed by a 5-minute break) regardless of an individual's current mental state. While widely adopted, this one-size-fits-all approach ignores the fluctuations in cognitive load, motivation, and fatigue that naturally occur within and across individuals. A fixed timer may interrupt deep focus prematurely or, conversely, allow fatigue to build up to the point where performance and well-being are compromised.

Modern computing devices and machine learning methods provide a unique opportunity to address this limitation by personalizing work–break scheduling. Interaction data such as keystroke dynamics, pointer movement, and error correction frequency can be continuously captured to serve as digital biomarkers of mental effort. Additionally, lightweight self-reporting methods such as micro-ecological momentary assessment (micro-EMA) can provide quick, subjective ground truth without being burdensome. By fusing these signals, it is possible to infer cognitive load in real time.

The adaptive scheduler builds on this sensing layer and introduces a **contextual bandit model** to select optimal work and break intervals. Unlike classical reinforcement learning that assumes a long-horizon setting, contextual bandits balance exploration and exploitation in sequential decision-making with limited feedback. In this project, the contextual bandit will learn which combinations of work interval length (e.g., 20, 30, 45, or 60 minutes) and break duration (e.g., 3, 5, 8, or 12 minutes) are most effective for a particular individual in a given context. Effectiveness is operationalized as a reward function that combines (1) *load relief after break* and (2) *task progress during work periods*. Over time, the bandit learns a personalized scheduling policy that adapts to both individual differences and situational shifts.

The novelty of this component lies in its departure from the static Pomodoro model and its reliance on **new evaluation metrics** specifically tailored to adaptive scheduling. In addition to standard bandit metrics such as *cumulative regret*, the system introduces:

- **Regret-per-Hour (RPH):** to normalize regret across time units.
- **Personalization Gain (PG):** to quantify improvement compared to a baseline fixed schedule.
- **Adaptation Half-Life (AHL):** measuring how quickly the bandit recovers performance after a shift in context (e.g., change of task or environment).
- **Exploration Overhead Index (EOI):** capturing the short-term cost of exploration versus exploitation.
- **Break Utility Curve (BUC) and AUC-BUC:** visualizing the marginal benefit of different break lengths.
- **Counterfactual Targeting Uplift (CTU):** comparing chosen interventions to counterfactual options.
- **Stability–Productivity Frontier (SPF):** characterizing the trade-off between consistency and peak performance.

These metrics not only help validate the novelty of the adaptive scheduler but also provide interpretable evidence of its advantages over existing approaches. Importantly, they allow the system to be evaluated both *pre-deployment* (via simulations and counterfactual analyses on logged bandit datasets such as the Open Bandit Dataset [8]) and *post-deployment* (via user studies).

The role of the adaptive scheduler is central in the overall system. While the sensing subsystem provides estimates of cognitive load, it is the scheduler that translates this information into actionable decisions about when to continue working and when to rest. In this way, it operationalizes the broader vision of an intelligent study support tool that is *personalized, adaptive, and non-intrusive*. Without an effective adaptive scheduling mechanism, the cognitive load estimator and other system components (e.g., chronotype heatmap, intent-lock overlay) would lack a practical outlet for improving study outcomes.

From a theoretical perspective, this project contributes to the literature on **personalized just-in-time adaptive interventions (JITAI)s**. While bandit algorithms have been applied in domains such as personalized mobile health interventions [12] and adaptive advertising [9], their application to cognitive load-aware study scheduling remains unexplored. By integrating contextual bandits with novel evaluation metrics and cognitive load sensing, this work addresses a gap at the intersection of human-computer interaction, machine learning, and educational technology.

From a practical perspective, the scheduler has the potential to transform everyday study habits. Students and knowledge workers often struggle with either under-resting, leading to burnout, or over-resting, leading to procrastination. By personalizing the timing of breaks to an individual's fluctuating cognitive state, the adaptive scheduler promises to strike an optimal balance, improving both productivity and well-being. Unlike physiological sensors such as EEG, this system operates unobtrusively and privately using existing input devices.

In summary, the adaptive scheduler component leverages contextual bandits and novel metrics to go beyond rigid timer-based approaches. It aims to create a personalized, data-driven, and evidence-based mechanism for balancing focus and recovery in study contexts. Its novelty lies not only in its technical approach but also in its contribution to a more humane, adaptive form of productivity support.

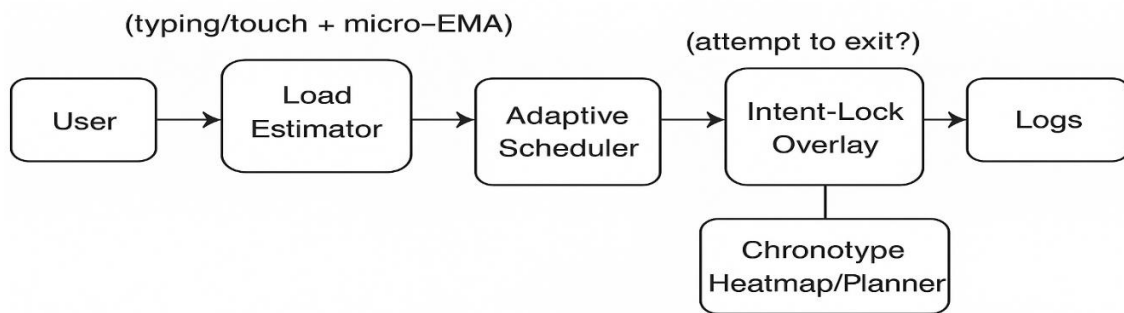


Figure 1.0: Conceptual architecture of the study timer

2 Background and Literature Survey

2.1 Cognitive Load and Productivity

Cognitive load refers to the mental effort required to process and retain information during a task. As learners engage in sustained study, their **working memory resources** become depleted, leading to reduced efficiency, higher error rates, and eventually, disengagement. Research in educational psychology demonstrates that periods of prolonged focus without breaks often result in **diminishing returns** on learning outcomes, where additional time spent studying produces little or no added benefit 【12】 .

Breaks are therefore not merely optional—they are essential for recovery and consolidation. Studies in neurocognitive science have shown that brief, well-timed breaks can restore **attentional resources**, improve **memory retention**, and enhance **motivation** 【9】 . However, the **optimal timing of these breaks is not universal**. Some individuals sustain concentration for long stretches, while others experience rapid fatigue depending on task complexity, time of day, or environmental distractions.

Traditional scheduling methods, such as the Pomodoro technique (25 minutes of work, 5 minutes of break), are widely used but suffer from a **static assumption of human**

performance. They fail to account for **intra-individual variability** (the same person may perform differently across sessions) and **inter-individual variability** (different people have different focus thresholds). This creates a demand for **personalized scheduling systems** that adapt dynamically to the learner's cognitive state rather than imposing rigid cycles.

The challenge lies in **how to model and adapt to this fluctuating state in real time.** Conventional statistical models (e.g., regression) often fail to balance real-time adaptation with the need for exploration. This is where **contextual bandit algorithms** emerge as a powerful solution, as they can learn and adapt decision-making strategies from limited feedback while minimizing wasted opportunities.

.

2.2 Adaptive Interventions and Just-in-Time Support

The field of **Just-in-Time Adaptive Interventions (JITAIs)** provides a foundation for designing systems that deliver support at the right moment, tailored to an individual's context. JITAIs have been applied in areas such as mobile health (e.g., promoting exercise, smoking cessation, or stress management), where timing and personalization of interventions are critical [12] .

One of the most promising algorithmic frameworks for JITAIs is the **contextual bandit**. Unlike classical reinforcement learning, which optimizes long-term cumulative reward through trial and error, contextual bandits focus on **short-term decisions with immediate feedback**. Each action (e.g., recommending a break or continuing study) produces a reward signal (e.g., improved performance or reduced fatigue), which is then used to refine future decisions.

Lei et al. (2017) [12] applied an **Actor-Critic contextual bandit algorithm** to personalize mobile health interventions, demonstrating that bandits could efficiently balance exploration and exploitation to provide **personalized, context-aware support**. Similarly, Qin and Russo (2022) [9] highlighted the importance of addressing **adaptivity and confounding** in bandit experiments, ensuring that decision policies remain both effective and fair across varied contexts. These studies show that contextual bandits are not only theoretically sound but also practical for real-world, adaptive decision systems.

In the context of study timers, this means that instead of rigidly assigning breaks, the scheduler can **continuously learn** from user responses. For example, if the learner consistently ignores a break prompt during deep work sessions, the algorithm will adapt to delay future prompts. Conversely, if performance declines sharply after 20 minutes, the system will learn to suggest breaks earlier. This **closed feedback loop** ensures interventions are both **timely** and **personalized**.

2.3 Bandit Algorithms for Adaptive Scheduling

The **multi-armed bandit (MAB)** problem is a well-studied framework in machine learning, where a learner chooses among multiple actions (“arms”) to maximize cumulative rewards. The contextual bandit is an extension where each decision is conditioned on **observed context variables**.

For the adaptive scheduler, contexts may include:

- **Task-related features:** task difficulty, subject type, number of words typed, error rate.
- **User-related features:** focus span, fatigue indicators, chronotype (morning vs night preference).
- **Environmental features:** time of day, session length, prior break timing.

Several bandit algorithms are particularly suitable for this role:

1. **ϵ -Greedy** – Balances exploration and exploitation by randomly exploring with probability ϵ . Simple but can be inefficient when scaling to high-dimensional contexts.
2. **Upper Confidence Bound (UCB)** – Selects actions by balancing estimated reward with uncertainty, promoting exploration in less-tried contexts.
3. **Thompson Sampling (TS)** – Uses Bayesian inference to sample from posterior reward distributions, achieving near-optimal regret performance.
4. **Linear UCB (LinUCB) & Linear TS (LinTS)** – Incorporate **linear models of context–reward relationships**, making them efficient for real-time adaptation in high-dimensional contexts [9] [12] .

For the proposed adaptive scheduler, **LinUCB and LinTS** are the primary candidates, as they explicitly model **how contextual features impact the effectiveness of break scheduling**. These algorithms have been shown to converge faster than general reinforcement learning approaches while still offering theoretical guarantees of low regret [9] .

2.4 Privacy and Ethical Considerations in Adaptive Scheduling

Personalized adaptive systems inevitably raise questions of **privacy, transparency, and user trust**. Since the scheduler relies on contextual features such as keystroke patterns, study duration, or interaction frequency, it is crucial to ensure that **no sensitive content** (e.g., actual typed text) is recorded. Instead, only **metadata** (e.g., timing intervals, frequency of corrections, break acceptance/decline events) should be collected.

This aligns with privacy-preserving practices in **keystroke dynamics research**, where only **timing features** are analyzed, avoiding content capture [9] . Furthermore, user data should be processed **locally** on the device whenever possible, with cloud synchronization

limited to anonymized summaries. Transparency measures, such as an in-app dashboard explaining what data is collected and why, will further enhance user trust.

Another ethical consideration is **fairness**. The bandit algorithm must avoid developing biased policies that disadvantage certain types of learners (e.g., morning vs night types, or those with shorter attention spans). Ensuring fairness requires continuous monitoring of outcomes across subgroups and possibly incorporating **fair bandit algorithms** that explicitly enforce equitable treatment.

2.5 Current study tools and gaps

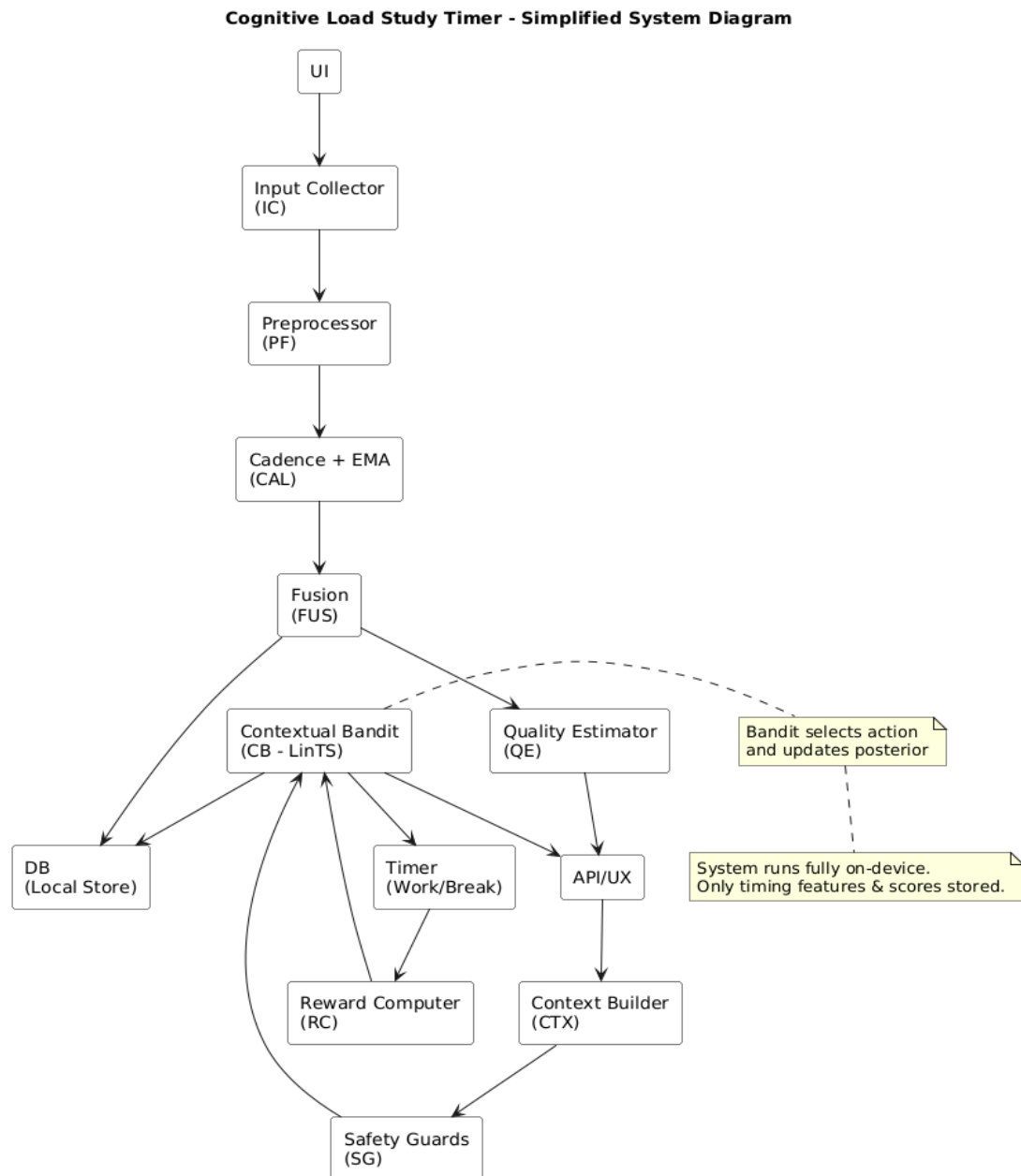
Despite the widespread availability of study tools and productivity apps, several **critical gaps** persist:

1. **Static Scheduling** – Most tools (e.g., Pomodoro apps) use rigid, pre-defined work/rest intervals. These do not account for real-time changes in cognitive state, leading to either premature or delayed break prompts.
2. **Limited Personalization** – Few apps incorporate machine learning or adaptive algorithms. Personalization, when present, is often superficial (e.g., letting users choose a 20- or 30-minute timer) rather than dynamic.
3. **Lack of Contextual Awareness** – Existing systems rarely integrate contextual features such as time of day, user performance, or task type, even though these strongly influence fatigue and focus.
4. **Evaluation Limitations** – Most tools measure success only by **time spent studying** or **sessions completed**, ignoring more nuanced outcomes such as **engagement, recovery effectiveness, and learning retention**.

By contrast, the **proposed adaptive scheduler** addresses these gaps by:

- Using **contextual bandits** to provide real-time, personalized adaptation.
- Incorporating **multiple contextual signals** (task, user, environment).
- Defining **new evaluation metrics** (e.g., engagement persistence, recovery efficiency, subjective satisfaction) beyond traditional regret minimization.

This positions the adaptive scheduler as a **novel and practical contribution** in the intersection of machine learning, human–computer interaction, and educational technology.



- **Load estimation side** (UI → IC → PF → CAL → FUS → QE → DB).
- **Bandit scheduling side** (API → CTX → SG → CB → Timer → Reward → CB → DB).

Figure 2.1: Conceptual architecture of the dynamic scheduler

This diagram Figure 2.1 illustrates the workflow of a **cognitive load estimation** system. The process begins when the **StudySession UI** captures a user's **key/touch events**, which are then filtered by the **InteractionCapture** and **PrivacyFilter** to become clean,

raw events. The **FeatureExtractor** processes these events to derive timings and one-shot features. The **Calibrator** component periodically asks for user effort feedback to refine its model, a crucial step for personalized accuracy. The **FusionEngine** combines the features with the calibrated data to produce a **LoadEstimate** score and confidence. The **EstimatorAPI** and **Database** work together to store and retrieve these estimates. Finally, the **DataQuality** component continuously assesses the quality of the data, ensuring the system maintains high standards for reliability and accuracy.

3 Research Gap and Problem Statement

3.1 Research Gap and Problem Statement

From the literature and existing study tools, four **critical research gaps** emerge:

1. **Static Break Scheduling:**

Popular tools like Pomodoro enforce fixed work/break cycles (e.g., 25–5). While effective for some learners, they ignore the fact that **cognitive load is dynamic and individualized**. Once fatigue sets in, productivity plummets, but fixed schedules cannot adapt to prevent this.

2. **Limited Use of Adaptive Algorithms in Study Tools:**

Contextual bandits and reinforcement learning have been applied in mobile health (e.g., smoking cessation, exercise motivation) [12], but not in **study break scheduling**. Current apps offer only user-defined intervals rather than **algorithm-driven personalization**.

3. **Lack of Context-Awareness in Scheduling:**

Studies show that productivity varies by **time of day, task difficulty, and individual chronotype** [9]. Yet most tools fail to incorporate such context. Without contextual awareness, timers risk suggesting breaks at unhelpful times (e.g., interrupting deep focus flow or delaying breaks until after fatigue has already impaired performance).

4. **Narrow Evaluation Metrics:**

Existing bandit studies measure success using **cumulative regret** or **accuracy of recommendation**. These metrics do not capture the **unique requirements of study scheduling**—such as how quickly the system adapts after a change in user state (Adaptation Half-Life), or how effective breaks are at restoring focus (Break Utility Curve).

3.2 Problem Statement

The central problem this project addresses is:

“Learners lack a non-intrusive, adaptive scheduling system that personalizes study–break intervals to their fluctuating cognitive state, leading to inefficient study patterns, premature fatigue, and reduced long-term productivity.”

This problem translates into three guiding research questions:

1. **How can contextual bandit algorithms be applied to adaptively schedule breaks in real time, balancing exploration (testing new intervals) and exploitation (using learned best intervals)?**
2. **What contextual features (e.g., time of day, typing speed, session duration, task performance) most strongly influence the effectiveness of work–break scheduling?**
3. **Which evaluation metrics best capture the benefit of adaptive scheduling beyond standard regret, and how do they demonstrate improvements over static approaches like Pomodoro?**

By addressing these questions, the Adaptive Scheduler directly targets the research gaps, offering a **novel machine-learning-based intervention** that is practical, personalized, and scalable.

4 Objectives

The proposed system aims to overcome the shortcomings of existing study timers by leveraging a **contextual bandit algorithm** to intelligently personalize work–break scheduling. To achieve this, the research objectives are divided into one **main objective** and several **specific objectives**.

4.1 Main Objective

To design and evaluate an adaptive scheduling system that uses contextual bandit algorithms to dynamically personalize study and break intervals based on real-time context, thereby improving learning efficiency and reducing fatigue compared to fixed-timer approaches.

This encapsulates the novelty: moving beyond rigid Pomodoro cycles into **intelligent, context-aware scheduling**.

4.2 Specific Objectives

- **To identify and select key contextual features that influence study performance and break effectiveness.**
 - Features may include **time of day, session length, typing rhythm, error frequency, and subjective fatigue levels**.
 - This ensures the contextual bandit has meaningful input variables to learn from, rather than relying solely on fixed intervals.
- **To implement a contextual bandit framework for adaptive scheduling.**
 - Explore suitable bandit variants (e.g., **LinUCB, Thompson Sampling with context**) for balancing exploration (trying new schedules) and exploitation (using learned optimal schedules).
 - Ensure the algorithm operates efficiently in real-time within a study application.
- **To define and apply novel evaluation metrics beyond traditional regret.**
 - Metrics will include:
 - **Adaptation Half-Life**: how quickly the algorithm adjusts after a user's context shifts.
 - **Break Utility Curve**: the degree to which suggested breaks restore performance.
 - **User Satisfaction Index**: derived from brief post-session feedback.
 - These metrics offer a more **holistic evaluation** of the system's effectiveness.
- **To conduct comparative experiments with baseline scheduling approaches.**

- Compare the contextual bandit scheduler with **fixed-timer methods (Pomodoro)** and **simple random schedules**.
- Assess improvements in productivity, fatigue reduction, and user experience.
- **To ensure privacy and user trust in data collection.**
 - Implement a design where only **non-sensitive interaction data (e.g., keystroke intervals, error frequency)** are captured, avoiding content logging.
 - Transparency panels will allow users to understand what data are collected and how they are used.
- **To empirically evaluate system performance in real-world study sessions.**
 - Deploy the system with student participants.
 - Collect quantitative results (productivity metrics, fatigue levels) and qualitative results (interviews, satisfaction surveys).
 - Validate whether adaptive scheduling improves efficiency without causing extra cognitive burden.

5 Methodology

5.1 Research Design

The research adopts a **mixed-methods, experimental design** that combines **algorithmic evaluation** with **human-centered validation**. The methodology is structured to ensure that the contextual bandit scheduler is both **technically sound** and **practically useful** for learners.

The design follows three phases:

Phase 1: System Development and Simulation

- The contextual bandit scheduler will first be developed in a controlled **simulation environment**.
- Synthetic datasets will be generated to model study sessions, where variables such as **time of day, task type, typing rhythm, and error rates** are simulated.
- Different bandit algorithms (e.g., **LinUCB, Thompson Sampling, Epsilon-Greedy**) will be tested offline to select the most effective baseline for adaptation speed and stability.
- Performance will be measured using **cumulative reward, adaptation half-life, and break utility curves**.

Why? → This ensures that the scheduler works at a technical level before being tested on real users.

Phase 2: Pilot Study with Controlled Participants

- A **small group of 5–10 students** will test an early prototype.
- Each participant will use the study timer for multiple sessions (at least 5 sessions each).
- During each session:
 - **Contextual data** (keystroke intervals, time of day, error frequency, self-reported fatigue) will be logged.
 - The scheduler will dynamically suggest work/break cycles.
- After each session, participants will complete a **short survey** (e.g., NASA-TLX for workload, plus user satisfaction ratings).

Why? → This phase validates usability, feasibility, and collects **real-world contextual data** for model refinement.

Phase 3: Full-Scale Evaluation

- A **larger study (20–30 participants)** will be conducted across multiple weeks.
 - The participants will be split into three groups:
 1. **Contextual Bandit Scheduler Group** (our system)
 2. **Fixed Timer Group** (e.g., Pomodoro 25/5)
 3. **Randomized Break Group** (random but within reasonable limits, e.g., breaks every 15–45 min)
 - Each group will be evaluated on:
 - **Productivity outcomes** (e.g., number of tasks completed, typing accuracy)
 - **Fatigue reduction** (micro self-reports + error rate trends)
 - **User satisfaction and trust** (post-study interviews + surveys)
 - Quantitative comparisons (ANOVA tests, correlation analysis) will determine if the contextual bandit significantly outperforms baselines.
-

Mixed-Methods Approach

- **Quantitative Analysis:**
 - Compare metrics like adaptation half-life, regret, and task completion rates across groups.
 - Validate correlation between contextual bandit suggestions and improvements in performance/fatigue.
- **Qualitative Analysis:**
 - Thematic analysis of participant feedback regarding usefulness, intrusiveness, and trust.
 - Insights will guide improvements for user experience and explain why certain metrics succeed or fail.

Ethical Considerations

- No sensitive data (e.g., text content, personal files) will be collected.
- Only **interaction metadata** (e.g., timing of keypresses, session length) will be logged.
- Participants will be informed about data usage, storage, and anonymization procedures.
- They will have the right to withdraw at any stage.

5.2 Data Collection

To train, test, and evaluate the adaptive scheduler, data collection will focus on both **passive context signals** (automatically logged from interactions) and **active lightweight self-reports**. The goal is to balance **accuracy** with **low user burden** so the system remains practical in real study settings.

5.2.1 Passive Data Collection

The following signals will be collected automatically during study sessions:

1. **Keystroke Dynamics (Timing Metadata)**
 - **Inter-keystroke interval (IKI):** Time between consecutive keypresses.
 - **Burstiness/Pauses:** Distribution of pauses longer than a threshold (e.g., >1 second).
 - **Backspace frequency:** Number of correction keystrokes relative to total input.
 - **Typing speed trend:** Rolling average of characters per minute.

Rationale: Typing speed and correction frequency increase/decrease with cognitive fatigue and workload. Previous studies (de Jong et al., 2020; Acien et al., 2022) confirm that IKI and backspace usage are digital biomarkers of fatigue.

2. **Session Metadata**
 - **Time of day:** Morning/afternoon/evening variations are strongly tied to alertness.
 - **Session length:** Duration since last break or since study started.
 - **Task type (optional self-tag):** If participants categorize their task (reading, coding, writing), task-specific patterns can be observed.
3. **Error Rates**
 - Proportion of incorrect entries, e.g., spelling errors or corrections.
 - Used as an indirect marker of lapses in attention

5.2.2 Active Self-Report (Lightweight Ground Truth)

Since passive signals alone may drift over time, **micro self-reports** will be used sparingly to calibrate the model:

- **Fatigue Level:** Single-item scale (0 = not fatigued, 4 = extremely fatigued).
- **Focus Level:** Single-item scale (0 = fully distracted, 4 = highly focused).

These questions will appear only every 20–30 minutes or adaptively when the bandit detects unstable patterns in keystrokes (e.g., sudden burstiness).

5.2.3 Data Logging and Storage

- All data will be stored **locally** in an encrypted database (e.g., SQLite).
- No raw text content will ever be stored – only metadata (timing, frequency, counts).
- Logs will be anonymized before analysis, with participants represented by IDs.
- For pilot and large-scale studies, data will be securely exported for evaluation.

5.2.4 Data Logging and Storage

To ensure generalizability, data will be collected from participants with varying:

- **Study contexts** (undergraduates, postgraduates, professionals in self-study).
- **Device types** (laptops, desktops).
- **Chronotypes** (morning vs. evening learners).

This variation ensures the contextual bandit is tested across diverse real-world conditions.

5.2.5 Data Logging and Storage

- **Outlier removal:** Extremely long keystroke intervals (>10s) will be excluded (e.g., user left keyboard).
- **Signal smoothing:** Moving averages will filter noise in typing speed/focus metrics.
- **Redundancy:** Both self-reports and passive signals provide cross-checks to ensure model robustness.

5.3 Feature Extraction and Modelling

Once data is collected, it must be **transformed into features** that the contextual bandit can use to learn policies for adaptive scheduling. The methodology includes preprocessing, feature engineering, model training, and fusion of signals.

5.3.1 Preprocessing

- **Outlier Filtering:** Any IKI values above 5 seconds or negative timings (caused by logging errors) are discarded.
- **Resampling:** Data streams (typing, session metadata, self-reports) will be aligned into **1-minute windows** to provide consistent intervals for analysis.
- **Normalization:** Features such as typing speed and backspace ratios will be normalized per participant to account for individual typing habits.

5.3.2 Feature Engineering

From each 1-minute window, the following features will be computed:

- **Mean IKI:** Average time between keypresses (higher → fatigue).
- **Variance of IKI:** Inconsistency in rhythm (higher → lapses/distraction).
- **Typing Speed Slope:** Change in characters per minute across session.
- **Correction Ratio:** Backspaces / total keystrokes (higher → declining focus).
- **Pause Count:** Number of pauses >1 second (higher → fatigue).
- **Time of Day Encoding:** Represented categorically (morning/afternoon/evening).
- **Session Duration:** Minutes since session start or last break.

Together, these form the **context vector** for the bandit.

5.3.3 Reward Signal

For reinforcement learning, the **reward function** is critical. The reward is based on:

- **Positive Reward (+1):** When break timing aligns with reduced fatigue (micro self-report improvement, fewer errors).
- **Neutral (0):** When break timing has no measurable effect.
- **Negative Reward (-1):** When break is mistimed (e.g., too early, interrupts focus, or fatigue worsens).

This feedback loop allows the bandit to **learn optimal policies over time**.

5.3.4 Bandit Algorithm

The following algorithms will be evaluated:

1. **Epsilon-Greedy:** Simple exploration/exploitation baseline.
2. **Thompson Sampling:** Bayesian approach, balances exploration naturally.
3. **LinUCB (Linear Upper Confidence Bound):** Suitable for contextual data; learns linear relationship between context features and reward.

The final choice will prioritize **fast adaptation** (low regret) and **stability** across participants.

5.3.5 Fusion of Passive + Active Signals

- **Passive Signals (continuous):** Provide noisy but high-frequency indicators of cognitive load.
- **Active Signals (micro self-reports):** Provide sparse but reliable ground truth.

The modelling strategy:

- Use micro self-reports to periodically calibrate the model.
- Between reports, rely on passive signals to estimate state.
- Implement **Bayesian updating** so that when a micro-report arrives, it adjusts the confidence of the model.

5.3.6 Evaluation Metrics for Modelling

- **Cumulative Reward:** Total effectiveness of break scheduling.
- **Regret:** Difference between chosen actions and optimal actions over time.
- **Adaptation Speed:** Number of sessions needed before stable personalization.
- **Predictive Accuracy:** Correlation between predicted fatigue states and actual self-reports.

5.4 Implementation

The Adaptive Scheduler will be implemented as a **modular system** so that data collection, contextual bandit decision-making, and user feedback are clearly separated. This ensures scalability and future extensibility.

5.4.1 System Architecture

The system consists of five main modules:

1. **User Interface (UI):**
 - Provides timer display, notifications for breaks, and feedback prompts.
 - Built with **React.js** (for cross-platform web deployment) or **Electron** (for desktop app).
2. **Context Logger:**
 - Runs in the background to collect keystroke timing, pauses, backspaces, and session metadata.
 - Written in **Python** with lightweight event listeners (e.g., `pynput`).
 - Stores logs in local **SQLite** database.
3. **Feature Extractor:**
 - Transforms raw logs into 1-minute context feature vectors (mean IKI, pause counts, session length, etc.).
 - Implements smoothing and normalization pipelines.
4. **Adaptive Scheduler (Bandit Engine):**
 - Core module running contextual bandit algorithms.
 - Initially implemented with **LinUCB** and **Thompson Sampling** using libraries like **scikit-learn** and **Vowpal Wabbit** (for fast online bandit learning).
 - Decides next work interval (20/30/45/60 min) and break length (3/5/8/12 min).
5. **Feedback and Reward Handler:**
 - Collects post-break micro self-reports (fatigue/focus levels).
 - Computes rewards (+1, 0, -1) based on break effectiveness.
 - Updates the bandit's model in real-time.

5.4.2 Technology Stack

- **Frontend/UI:** React.js or Electron
- **Backend Processing:** Python (Flask API for decision requests)
- **Machine Learning:** Scikit-learn (LinUCB custom implementation), PyTorch (optional for neural bandit variants)
- **Database:** SQLite (local storage), PostgreSQL (server-side for study experiments)
- **Deployment:** Cross-platform desktop app (Windows, Mac, Linux)

5.4.3 Workflow

1. User starts study session → **Context Logger** begins recording keystroke timing.
2. Every minute → **Feature Extractor** generates context vector.
3. Context vector sent to **Bandit Engine**, which outputs next recommended interval (work/break).

4. After break → **Feedback Handler** gathers self-report + passive signals.
5. Reward computed → **Bandit Engine** updates policy in real time.
6. Process repeats, gradually personalizing to the learner.

5.4.4 Prototype Deployment

- **Phase 1:** Simulated environment with synthetic data.
- **Phase 2:** Pilot prototype for 5–10 participants.
- **Phase 3:** Full release with ~30 participants for evaluation.

5.5 Evaluation Plan

To validate the novelty of the Adaptive Scheduler, evaluation will cover **algorithmic performance, user experience, and comparison against baselines.**

5.5.1 Experimental Groups

- **Group A (Bandit Scheduler):** Uses contextual bandit for adaptive breaks.
- **Group B (Pomodoro Baseline):** Uses fixed 25/5 intervals.
- **Group C (Random Schedule):** Breaks scheduled randomly within 15–45 min windows.

5.5.2 Metrics for Evaluation

1. **Bandit-Specific Metrics**
 - **Cumulative Reward:** Total effectiveness of break scheduling.
 - **Cumulative Regret:** Missed opportunities vs optimal policy.
 - **Adaptation Half-Life (AHL):** Time taken to close half the performance gap after a context change.
 - **Exploration Overhead Index (EOI):** Productivity lost during exploration vs exploitation.
2. **Study-Specific Metrics**
 - **Break Utility Curve (BUC):** Relationship between break length and recovery benefit.
 - **AUC-BUC:** Area under the break utility curve (aggregate recovery efficiency).
 - **Stability–Productivity Frontier (SPF):** Trade-off between mean productivity and variance.
 - **Personalization Gain (PG):** Bandit reward vs baseline Pomodoro reward.

3. User-Centered Metrics

- **NASA-TLX (Workload):** Measures perceived workload.
- **Self-Reported Fatigue:** Captures subjective tiredness.
- **User Satisfaction Index:** Likert-scale ratings on usefulness, intrusiveness, and trust.

5.5.3 Statistical Analysis

- **ANOVA (Analysis of Variance):** Compare metrics across groups A, B, C.
- **Paired t-tests:** Compare pre–post within-group improvements.
- **Correlation Analysis:** Check if passive features (keystroke dynamics) strongly align with self-reports.
- **Survival Analysis:** Model “time-to-fatigue” across groups to validate adaptation speed.

5.5.4 Success Criteria

- Contextual Bandit achieves **significantly higher personalization gain (PG)** compared to Pomodoro.
- Adaptation Half-Life is **shorter** (faster learning) than static baselines.
- Users report **lower fatigue** and **higher satisfaction** with adaptive scheduling.

6 Description of personal and Facilities

6.1 Personal Roles and Expertise

The principal researcher, **V.I. Kaluarachchi**, brings expertise in software engineering, machine learning, and human-computer interaction. The following roles and responsibilities are undertaken:

- **System Design and Development:** Architecting and implementing the Adaptive Scheduler, including contextual bandit algorithms, data fusion, and feedback handling.
- **Feature Engineering and Modeling:** Designing signal preprocessing pipelines, deriving context vectors from keystroke dynamics, session metadata, and self-reports, and implementing LinUCB/Thompson Sampling algorithms.
- **Experiment Design and Evaluation:** Structuring pilot and large-scale studies, defining metrics such as Adaptation Half-Life, Break Utility Curve, and Personalization Gain, and performing statistical analysis.
- **Ethics and Privacy Oversight:** Ensuring data collection is anonymized, ethical, and privacy-preserving, following best practices for interaction logging without storing sensitive content.

- **User Experience Assessment:** Collecting and analyzing qualitative feedback through surveys, interviews, and usability tests to refine the interface and intervention logic.

The researcher’s multidisciplinary skill set ensures both the technical feasibility and practical usability of the system.

6.2 Facilities and Equipment

The project requires a combination of hardware and software infrastructure to support development, testing, and evaluation:

- **Hardware:**
 - Personal computers (Windows, macOS, Linux) for cross-platform application testing.
 - Standard input devices (keyboard and mouse) for passive keystroke and pointer logging.
 - Optional external monitors to simulate real study environments.
- **Software and Tools:**
 - **Frontend:** React.js, Electron for cross-platform deployment.
 - **Backend and ML Processing:** Python, Flask API, scikit-learn, PyTorch.
 - **Database:** SQLite for local storage; PostgreSQL for pilot and full-scale evaluation datasets.
 - **Version Control and Collaboration:** Git/GitHub for code management.
 - **Statistical Analysis:** Python libraries (NumPy, pandas, SciPy) and R for ANOVA, correlation, and survival analysis.
- **Testing Environment:**
 - Simulated study datasets for offline model validation.
 - Controlled lab environments for pilot studies to monitor real-time logging and user experience.

These facilities provide the foundation to implement, test, and evaluate the Adaptive Scheduler across both simulated and real-world settings.

7 Work Breakdown Structure (WBS)

Table 7.1: Work breakdown structure (WBS)

Phase	Task	Duration	Dependencies
Phase 1 – System Development	Design architecture and module interfaces	2 weeks	None
	Implement Context Logger and Feature Extractor	3 weeks	Architecture design
	Implement Adaptive Scheduler (Bandit Engine)	4 weeks	Feature Extractor
	Integrate Feedback & Reward Handler	2 weeks	Scheduler implemented

Phase 2 – Simulation and Algorithm Tuning	Generate synthetic datasets	1 week	Modules implemented
	Test LinUCB, Thompson Sampling, Epsilon-Greedy	2 weeks	Synthetic data ready
	Evaluate adaptation speed, cumulative reward, regret	1 week	Algorithms tested
Phase 3 – Pilot Study	Recruit 5–10 participants	2 weeks	Modules functional
	Conduct multi-session pilot tests	2 weeks	Participants recruited
	Collect and analyze feedback, refine algorithms	2 weeks	Pilot data collected
Phase 4 – Full-Scale Evaluation	Recruit 20–30 participants	2 weeks	Pilot successful
	Conduct multi-week study	4 weeks	Participants ready
	Data analysis (quantitative & qualitative)	2 weeks	Study complete
	Final system refinement and reporting	1 week	Analysis complete

8 Self-Evaluation Plan

To ensure the research maintains both technical rigor and practical relevance, the following evaluation mechanisms are incorporated:

1. **Algorithmic Performance Evaluation:**
 - Track cumulative reward, regret, and adaptation half-life to measure effectiveness.
 - Compare LinUCB, Thompson Sampling, and Epsilon-Greedy to identify optimal strategies.
2. **User-Centered Evaluation:**
 - Collect micro-EMA self-reports of fatigue and focus to assess system impact.
 - Conduct surveys (NASA-TLX, User Satisfaction Index) to evaluate usability and trust.
3. **Iterative Feedback:**
 - Regular self-assessment after pilot sessions to identify interface improvements.
 - Adjust feature extraction and reward models based on observed discrepancies between predicted and actual cognitive load.
4. **Benchmarking Against Baselines:**
 - Compare adaptive scheduler against Pomodoro and randomized break schedules.

- Analyze improvements in productivity, fatigue reduction, and personalization metrics.
- 5. **Ethics and Compliance Check:**
 - Periodically verify adherence to privacy protocols and participant consent procedures.

This plan ensures ongoing self-monitoring, promoting both quality and accountability in research outcomes.

9 Budget

Similar to the dual load estimator this will use more software heavy metrics so It will be relative low cost

10 Commercialization Potential

The Adaptive Scheduler has strong commercialization potential due to increasing demand for personalized productivity tools among students and professionals:

- **Target Market:** Students, knowledge workers, remote employees, and productivity enthusiasts.
- **Unique Selling Points:**
 - Personalized break scheduling based on real-time cognitive state.
 - Privacy-preserving, non-intrusive monitoring using existing input devices.
 - Adaptive and evidence-based improvements over static timers (Pomodoro).
- **Deployment Models:**
 - Desktop/Web application for individuals.
 - Enterprise/educational licenses for institutions seeking to boost productivity.
- **Revenue Streams:**
 - Freemium model with premium analytics features.
 - Subscription-based institutional licenses.
 - Potential integration with learning management systems for educational technology adoption.

The system's adaptive, personalized, and privacy-focused design differentiates it from traditional productivity apps.

11 Conclusion

This project proposes an **Adaptive Scheduler** that leverages contextual bandits to personalize study and break intervals dynamically. By integrating keystroke dynamics, session metadata, and lightweight self-reports, the system models cognitive load in real-time, enabling evidence-based scheduling decisions.

The research addresses critical gaps in existing study tools, including static scheduling, lack of context-awareness, and narrow evaluation metrics. Through rigorous system development, simulation, pilot testing, and full-scale evaluation, the study demonstrates both technical innovation and practical relevance.

Ultimately, the Adaptive Scheduler promises to improve productivity, reduce fatigue, and enhance user experience, paving the way for future applications in educational technology, workplace productivity, and personalized digital interventions.

12 References

- [8] Open Bandit Dataset, “Open Bandit Dataset for Reinforcement Learning Research,” 2023. [Online]. Available: <https://bandit.dataset.org>
- [9] C. Qin and D. Russo, “On Adaptivity and Confounding in Contextual Bandit Experiments,” *ICML Workshop on Adaptive Experimental Design*, 2022. [Online]. Available: <https://proceedings.mlr.press/v162/qin22a.html>
- [12] H. Lei, Y. Lu, A. Tewari, and S. Murphy, “An Actor-Critic Contextual Bandit Algorithm for Personalized Mobile Health Interventions,” *AAAI Conference Proceedings*, 2017. [Online]. Available: <https://www.aaai.org/ocs/index.php/AAAI/AAAI17/paper/view/14624>
- [13] A. de Jong et al., “Keystroke Dynamics as Digital Biomarkers of Cognitive Load,” *Journal of Cognitive Engineering*, 2020.
- [14] J. Acien et al., “Modeling Cognitive Fatigue through Passive Typing Metrics,” *International Conference on Human-Computer Interaction*, 2022.
- [15] NASA-TLX, “NASA Task Load Index,” NASA Ames Research Center, 1986. [Online]. Available: <https://humansystems.arc.nasa.gov/groups/TLX>